DAY 29

Project Review, Quality Audit, Risk Review, and Final Demonstration Preparation

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Conduct a structured review of project scope, requirements, design, implementation, and testing evidence.

• Distinguish a project review from a formal audit and a product demonstration.

• Evaluate software quality using defined criteria and evidence rather than opinion alone.

• Identify, assess, prioritize, and respond to project risks.

• Prepare a reliable final demonstration, handover, and list of known limitations.

• Answer review questions clearly and transparently.

Running case study: The College Event Management System (CEMS) supports event browsing and registration. The team is preparing for a pilot release and must determine whether the product is ready, what evidence supports that decision, and what risks remain.

1. Why Review a Project Before Delivery?

A project review is a structured checkpoint to understand what has been delivered, whether it meets the agreed goals, what evidence supports quality, and what still needs attention. It helps a team find gaps before users discover them in production.

A review is not simply a meeting where the team says the project is finished. The team should compare actual results with agreed requirements, schedule, scope, quality expectations, and release criteria. Where evidence is missing, record the gap instead of assuming success.

Review area	Questions to ask	Possible evidence
Scope and requirements	Were the agreed requirements addressed? Have changes been approved?	Requirements checklist, acceptance criteria, change records
Design and architecture	Does the implementation follow the intended design? Are important decisions documented?	Architecture diagram, design notes, code review
Implementation	Is the work integrated and maintainable? Are important changes reviewed?	Repository history, pull requests, build results
Testing	Have relevant normal, boundary, negative, and regression cases been checked?	Test cases, execution results, defect reports
Project management	Are milestones, dependencies, and open work visible?	Jira board, schedule, risk register
Release and support	Can the product be deployed, used, and supported safely?	Release checklist, user guide, rollback/support plan

2. Review, Audit, and Demonstration: What Is the Difference?

Activity	Main purpose	Typical output
Project review	Assess progress, decisions, results, and remaining work	Actions, decisions, updated status, and owners
Quality audit	Check whether defined processes or standards were followed, using objective evidence	Findings, evidence, nonconformities, and corrective actions
Final demonstration	Show the product's behaviour to an audience using representative scenarios	Feedback, questions, acceptance discussion, and follow-up actions

Activity	Main purpose	Typical output
Retrospective	Reflect on how the team worked and identify improvements	Agreed process improvements for future work

A classroom review is not automatically a formal independent audit. An audit normally has defined criteria, evidence collection, and an appropriate level of independence. Use the term that matches the activity actually performed.

3. Create a Requirements-to-Evidence Checklist

Traceability connects each important requirement to evidence that it has been designed, implemented, and tested. A checklist helps reviewers avoid relying only on memory or a polished presentation.

Requirement	Implementation evidence	Verification evidence	Status example
Student can view published events	Event list and details screen	Test for published and draft events	Pass / investigate
Student can register if a seat remains	Registration API and UI	Successful registration test	Pass / investigate
Duplicate registration is prevented	Duplicate check and data protection	Repeated registration test	Pass / investigate
Full event rejects new registration	Server-side capacity enforcement	Full-capacity and boundary tests	Pass / investigate
Student can retrieve registration	Registration lookup screen/API	Sign out/in or new-session retrieval test	Pass / investigate

> Use real results in the Status column. Do not mark a requirement as passed merely because the code appears to exist. A requirement can be implemented but unverified, partially met, or blocked by an unresolved defect.

4. Quality Review: Criteria and Evidence

Software quality is multidimensional. A system may look attractive but fail under load, or pass functional tests but be difficult to maintain. The team should select quality criteria that fit the product and its risks.

Quality concern	Review question	Example evidence
Functional suitability	Does the feature do what users need?	Acceptance tests and scenario results
Reliability	Does it behave consistently, including at boundaries?	Repeated runs, error handling, capacity tests
Usability and accessibility	Can intended users understand and operate it?	Task-based user feedback, accessibility checks
Performance	Does it respond within an agreed target under expected conditions?	Measurements from a documented test setup
Security and privacy	Are access, data handling, and permissions appropriate?	Access-control tests, review findings, data-handling checks
Maintainability	Can the team understand and change the implementation?	Code review, modularity, tests, documentation
Compatibility	Does it work in supported environments?	Browser/device or integration test results

Avoid unsupported claims such as “the system is secure” or “the application is fast.” Prefer evidence-based statements: “The three defined capacity scenarios passed in the test environment on this build.” State the scope and limits of the evidence.

5. Defect Triage and Corrective Action

A defect should be evaluated by its impact, likelihood, affected users, available workaround, and release context. Severity describes the impact of the defect; priority describes how urgently the team should address it. They are related but not identical.

Example defect	Possible severity / priority reasoning	Possible action
Users can register beyond event capacity	High impact on trust and event operations; likely release blocker	Fix server-side rule and retest boundary/concurrency cases
Confirmation email is delayed but registration is saved	Impact depends on user expectations and recovery path	Investigate timing; show confirmation on screen and monitor email
Small alignment issue on one page	Usually lower functional impact, though accessibility may change priority	Record, assess user impact, fix now or schedule
Organizer cannot publish any event	High operational impact for event owners	Prioritize investigation and verify end-to-end publishing

A corrective action should have an owner, due point or review date, and a way to verify effectiveness. Closing a defect ticket without retesting the affected behaviour is weak evidence that the problem has been resolved.

6. Risk Review

A risk is an uncertain event or condition that could affect project objectives. A risk differs from an issue: a risk may happen in the future; an issue is already happening and needs action.

A simple qualitative approach scores probability and impact from 1 to 5. The product of the scores can help rank risks, but the number is a prioritization aid—not a precise prediction.

Risk	Probability (1–5)	Impact (1–5)	Score	Response
Capacity race causes overbooking	3	5	15	Use server-side transaction/constraint strategy; test concurrent requests
Demo environment becomes unavailable	2	4	8	Rehearse, prepare local backup and screenshots
Requirements change near release	4	3	12	Confirm scope, assess impact, use change control
Key team member unavailable	3	3	9	Document setup, cross-train, share ownership
Sensitive data appears in test records	2	5	10	Use synthetic data and review data-handling practices

Risk response strategies

• **Avoid:** change the plan so the threat is removed.

• **Mitigate:** reduce probability or impact through preventive action.

• **Transfer/share:** shift or share some exposure through an appropriate agreement or service arrangement.

• **Accept:** consciously retain the risk, often with monitoring or a contingency plan.

Review risks throughout the project. A mitigation is not complete merely because it has been written down; the team should check whether it was carried out and whether the risk has changed.

7. Final Demonstration Preparation

A demonstration should prove a meaningful user journey, not merely show screens. Select a scenario that represents the main value of the project and prepare the data and environment in advance.

Example 5-minute CEMS demo

Time	Action	What to explain
0:00–0:30	Introduce the user problem	Manual registration and uncertain seat counts
0:30–1:15	Show the published event list	How students find relevant events
1:15–2:15	Open an event and register	Validation, confirmation, and registration reference
2:15–3:00	Try a duplicate or full event	How the system protects correctness

Time	Action	What to explain
3:00–4:00	Show organizer view or test evidence	How capacity and quality are checked
4:00–5:00	Summarize limitations and next steps	What is complete, what remains, and known risks

• Use synthetic accounts and safe test data.

• Rehearse the exact steps on the same build and environment where possible.

• Prepare a fallback if Wi-Fi, services, or the demo environment fail.

• Do not claim that a planned or mocked feature is fully implemented.

• Assign speaking and navigation responsibilities across the team.

8. Handover and Release Readiness

A handover enables another person to operate, maintain, or continue the project. It should contain enough information to use the product and understand its current limitations.

• Project purpose, scope, and supported user workflows.

• Repository location and setup/build instructions.

• Configuration requirements, with secrets stored securely outside the document.

• Test summary and known defects or limitations.

• Deployment and rollback instructions, if applicable.

• Support contact or ownership, where relevant.

• Release notes and future-work list.

Release readiness checklist: required acceptance criteria verified; critical defects resolved or explicitly accepted by the authorized decision-maker; build and tests recorded; user-facing documentation updated; deployment/support plan reviewed; known risks communicated.

9. Classroom Activity: Run a Mock Project Review

Form teams and assign reviewer, developer, tester, product representative, and project coordinator responsibilities. Use the CEMS project packet created in earlier sessions.

1. Select five important requirements and identify the evidence needed for each.

2. Review test results and mark each requirement as passed, failed, unverified, or not applicable.

3. Identify at least three defects or gaps and explain their severity and priority.

4. Score three project risks and propose an owner and response for each.

5. Rehearse a five-minute demonstration and prepare a fallback plan.

6. Make a release recommendation: ready, ready with accepted limitations, or not ready. Support it with evidence.

Deliverables: requirements evidence checklist, defect triage list, risk register, demo outline, and a written release recommendation.

10. Review Questions with Suggested Answers

1. Why conduct a project review before release?

Suggested answer: To compare results with agreed goals, inspect evidence, identify gaps, and make informed release decisions.

2. How does a review differ from an audit?

Suggested answer: A review assesses progress and outcomes; an audit checks conformance against defined criteria using objective evidence.

3. What is traceability?

Suggested answer: The ability to connect requirements to design, implementation, tests, and defects.

4. What is the difference between a risk and an issue?

Suggested answer: A risk is uncertain and may occur; an issue is already occurring.

5. How do severity and priority differ?

Suggested answer: Severity concerns impact; priority concerns urgency of handling.

6. Why is test evidence needed?

Suggested answer: It supports claims that the software meets expected behaviour and quality criteria.

7. What should a team do if a requirement is unverified?

Suggested answer: Record it as unverified, identify the missing evidence, assign follow-up work, and avoid claiming it passed.

8. Why prepare a demo fallback?

Suggested answer: To reduce the chance that environment or connectivity problems prevent the audience from seeing the intended workflow.

9. What belongs in a handover?

Suggested answer: Setup/use instructions, ownership, test summary, known issues, deployment/support information, and release notes as applicable.

10. What supports a release recommendation?

Suggested answer: Evidence from requirements checks, tests, defect status, risks, and operational readiness.

11. Glossary

Term	Meaning
Project review	Structured assessment of progress, outcomes, and remaining work
Quality audit	Evidence-based check against defined standards or process criteria
Traceability	Links between requirements and the artefacts that realize or verify them
Defect severity	Degree of impact caused by a defect
Defect priority	Urgency with which a defect should be addressed
Risk	Uncertain event or condition that could affect objectives
Mitigation	Action intended to reduce a risk's probability or impact
Release readiness	Evidence that a build is suitable for its intended release
Handover	Transfer of knowledge and operational responsibility
Known limitation	A documented constraint or shortcoming that remains in the product

12. Homework

1. Prepare a one-page quality review for a software project, with at least five criteria and evidence for each.
2. Create a risk register with five risks, probability, impact, response, and owner.
3. Write a release recommendation for CEMS and justify it using test results, open defects, and known limitations.
4. Prepare a five-minute demo script and a fallback plan.
5. List the information another team would need to build, test, deploy, and maintain your project.

Next chapter: Day 30 — Final Project Presentation, Viva Preparation, Lessons Learned, and Course Consolidation.